

一、名词术语解释：（20 分）

1. 分布式系统的特征（5 分）

答：（来自知识点）

并发性 - 程序通过共享资源并行

缺乏全局时钟 - 分布式系统中的计算机只能通过本地时钟交换消息；不可预测的消息延迟限制了准确性；没有一个全局时钟的概念。

故障独立性 - 计算机中的故障或程序中的异常马上不能被与之通讯的其他组件感知；系统设计者需要为故障处理做计划。

2. 分布式透明性（5 分）

答：（来自网络）

透明性是指每一个分布节点对用户的应用来说都是透明的，看不出是本地还是远程。在 分布式、系统中，用户感觉不到是分布的，即用户不须知道关系是否分割、有无副本、数据存于哪个站点以及事务在哪个站点上执行等。

3. 逻辑时钟同步（5 分）

答：（来自网络）（同 2019.1）

逻辑时钟是分布式系统的特征，要求的是系统节点进程之间的相对一致性。只有相关的进程才需

要有逻辑时钟同步，同步的目的是维持时间的顺序性，除时间的基本特性外，与物理时钟之间没有

通用意义上的明确的关系。

4. 云计算与大数据（5 分）

答：（来自网络）（云计算 - 同 2017.7，2018.7，大数据 - 同 2017.7，2019.1）

云计算（cloud computing）- 是分布式计算的一种，指的是通过网络“云”将巨大的数据计算处理程序分解成无数小程序，然后，通过多部服务器组成的系统进行处理和分析这些小程序得到结果并返回给用户。云计算早期，简单地说，就是简单的分布式计算，解决任务分发，并进行计算结果的合并。因而，云计算又称为网格计算。通过这项技术，可以在很短的时间内（几秒钟）完成对数以万计的数据的处理，从而达到强大的网络服务。

大数据- 研究机构 Gartner 给出了这样的定义。“大数据”是需要新处理模式才能具有更强的决策力、洞察发现力和流程优化能力来适应海量、高增长率和多样化的信息资产。

麦肯锡全球研究所给出的大数据定义是：一种规模大到在获取、存储、管理、分析方面大大超出了传统数据库软件工具能力范围的数据集合，具有海量的数据规模、快速的数据流转、多样的数据类型和价值密度低四大特征。

二、问答题，(40 分)

1. 在分布式系统的同步或异步分析中，关注消息传递的时序关系，举例说明并图示不同网络结点间进程“事件的实时排序”。可能出现的异常情况，分析原因。(10 分)

答：(来自知识点)

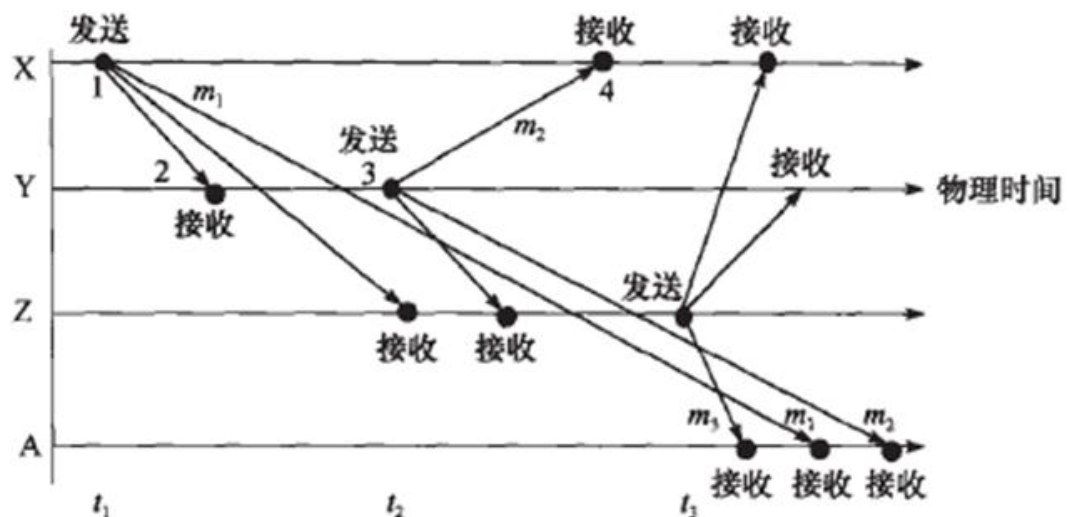


图 2-13 事件的实时排序

由于时钟不能精确同步，提出了逻辑时间的模型，因为逻辑上，我们知道消息在它发送之后才被接受，所以对上图所示的成对事件给出一个逻辑排序。如：X 在 Y 接受到 M_1 前发送 M_2 ，Y 在 X 接受到 M_2 前发送 M_2 。Y 在发送 M_2 前接收 M_1

2. 在分布式系统建模中，什么是场景？什么是逻辑视图？什么是开发视图？还有哪些视图？举例说明场景、视图在分布式系统建模中的重要关系？(10 分)

答：暂无

3. 什么叫远程过程调用？举例说明。描述远程过程调用 RPC 的工作原理和 RPC 调用的步骤。（10 分）

答：（来自网络）

远程过程调用

远程过程调用采用客户机/服务器(C/S)模式。请求程序就是一个客户机，而服务提供程序就是一台服务器。和常规或本地过程调用一样，远程过程调用是同步操作，在远程过程结果返回之前，需要暂时中止请求程序。使用相同地址空间的低权进程或低权线程允许同时运行多个远程过程调用。也就是说两台服务器 A，B，一个应用部署在 A 服务器上，想要调用 B 服务器上应用提供的函数/方法，由于不在一个内存空间，不能直接调用，需要通过网络来表达调用的语义和传达调用的数据。

工作原理

RPC 服务方通过 RpcServer 去导出（export）远程接口方法，而客户方通过 RpcClient 去引入（import）远程接口方法。

客户方像调用本地方法一样去调用远程接口方法，RPC 框架提供接口的代理实现，实际的调用将委托给代理 RpcProxy。

代理封装调用信息并将调用转交给 RpcInvoker 去实际执行。

在客户端的 RpcInvoker 通过连接器 RpcConnector 去维持与服务端的通道 RpcChannel，并使用 RpcProtocol 执行协议编码（encode）并将编码后的请求消息通过通道发送给服务方。

RPC 服务端接收器 RpcAcceptor 接收客户端的调用请求，同样使用 RpcProtocol 执行协议解码（decode）。

解码后的调用信息传递给 RpcProcessor 去控制处理调用过程，最后再委托调用给 RpcInvoker 去实际执行并返回调用结果。

调用步骤

- 1) 服务消费方（client）调用以本地调用方式调用服务；
- 2) client stub 接收到调用后负责将方法、参数等组装成能够进行网络传输的消息体；
- 3) client stub 找到服务地址，并将消息发送到服务端；

- 4) server stub 收到消息后进行解码；
- 5) server stub 根据解码结果调用本地的服务；
- 6) 本地服务执行并将结果返回给 server stub；
- 7) server stub 将返回结果打包成消息并发送至消费方；
- 8) client stub 接收到消息，并进行解码；
- 9) 服务消费方得到最终结果。

RPC 的目标就是要 2~8 这些步骤都封装起来，让用户对这些细节透明。

4. 在分布式环境下，什么是代码迁移？常见迁移的动机？举例说明。（10 分）

答：（同 2019.7）

（来自书）

代码迁移是指能从一台计算机发送到另一台计算机，并在目的计算机上运行的代码。

例子：虚拟机，提供了一种使代码可在任何计算机上运行的方法。

（来自网络）

随着代码变老，通常有必要对其进行现代化。有以下动机：

- 1)找到了一种更好的方法
- 2)需要出于支持/许可或仅出于最佳实践的原因而更新核心库/技术
- 3)需要在更现代的基础架构上运行该软件

三、处理器分配与死锁：（20 分）

在分布式环境下，讨论处理器任务分配与死锁处理。回答下列问题：

1. （5 分）举例说明分布式处理器任务分配。比较与单处理器分配的不同。

答：（来自网络）

分布式环境下的任务分配问题是一类典型的组合优化问题。应用蚁群算法来解决多处理器分布式系

统上的任务分配问题,一个任务只能分配给一个处理器处理,而一个处理器可以处理多个任务。

单处理机可以通过软件分配处理器运算资源，比如在一秒内同时分配给两个程序各 0.5 秒的运算时

间，在没有可调用资源的时候，程序被中断，单处理机可以来回兼顾，但是每个任务的效率都只有

二分之一。分布式环境同样依赖程序的任务分配，大多数程序能调用多线程，他们同时为一个任务运行，并行运算可以在程序内部同时处理两个线程的工作，效率接近加倍。

单处理机从宏观上是同时为多个进程服务的，而实际上是轮流为每个进程服务一段时间片，由于处理机的高速高频率，因此我们从宏观上是看不出处理机轮流工作的。这是分时处理。分布式环境由于拥有多个核心，因此能够在同一时间为多个进程服务，并不像单核那样要轮流分配时间片。因此能够更好的工作。

2. （5 分）比较进程调度与线程调度的异同？解释在分布式系统中的多线程并发处理。

答：（同 2018.7）

进程调度 - 操作系统管理了系统有限资源，当有多个进程（或多个进程发出的请求）要使用这些资源时，因为资源有限性，必须按照一定的原则选择进程（请求）来占用资源，这就是调度。

线程调度 - 指按照特定机制为多个线程分配 CPU 的使用权。

区别 -

- 1) 地址空间：进程内的一个执行单元；进程至少有一个线程；它们共享进程的地址空间；而进程有自己独立的地址空间；
- 2) 资源拥有：进程是资源分配和拥有的单位， 同一个进程内的线程共享进程的资源
- 3) 线程是处理器调度的基本单位，但进程不是。
- 4) 二者均可并发执行。

进程和线程都是由操作系统所体会的程序运行的基本单元，系统利用该基本单元实现系统对应用的并发性

进程和线程的区别在于：

简而言之，一个程序至少有一个进程，一个进程至少有一个线程。

线程的划分尺度小于进程，使得多线程程序的并发性高。

另外，进程在执行过程中拥有独立的内存单元，而多个线程共享内存，从而极大地提高了程序的运行效率。

线程在执行过程中与进程还是有区别的。每个独立的线程有一个程序运行的入口、顺序执行序列和程序的出口。但是线程不能够独立执行，必须依存在应用程序中，由应用程序提供多个线程执行控制。从逻辑角度来看，多线程的意义在于一个应用程序中，有多个执行部分可以同时执行。但操作系统并没有将多个线程看做多个独立的应用，来实现进程的调度和管理以及资源分配。这就是进程和线程的重要区别。

进程是具有一定独立功能的程序关于某个数据集合上的一次运行活动，进程是系统进行资源分配和调度的一个独立单位。

线程是进程的一个实体，是 CPU 调度和分派的基本单位，它是比进程更小的能独立运行的基本单位。线程自己基本上不拥有系统资源，只拥有一点在运行中必不可少的资源 (如程序计数器，一组寄存器和栈)，但是它可与同属一个进程的其他的线程共享进程所拥有的全部资源。

一个线程可以创建和撤销另一个线程；同一个进程中的多个线程之间可以并发执行。

(来自网络)

多线程并发处理：指从软件或者硬件上实现多个线程并发执行的技术。具有多线程能力的计算机因有硬件支持而能够在同一时间执行多于一个线程，进而提升整体处理性能。具有这种能力的系统包括对称多处理机、多核心处理器以及芯片级多处理（Chip-level multithreading）或同时多线程（Simultaneous multithreading）处理器。在一个程序中，这些独立运行的程序片段叫作“线程”（Thread），利用它编程的概念就叫作“多线程处理（Multithreading）”。具有多线程能力的计算机因有硬件支持而能够在同一时间执行多于一个线程（台湾译作“执行绪”），进而提升整体处理性能。

3. （5分）如何进行分布式系统的“负载均衡”处理？

答：（来自网络）

分布式系统中，服务请求的负载均衡是指，当处理大量用户请求时，请求应尽量均衡地分配到多台服务器进行处理，每台服务器处理其中一部分而不是所有的用户请求，以完成高并发的请求处理，避免因单机处理能力的上限，导致系统崩溃而无法提供服务的问题。

通常情况下，负载均衡可以分为两种：

一种是请求负载均衡，即将用户的请求均衡地分发到不同的服务器进行处理；

另一种是数据负载均衡，即将用户更新的数据分发到不同的存储服务器。

常见的负载均衡策略主要包括轮询策略、随机策略、哈希和一致性哈希等策略。

实现方式有多种。大致可以分为以下几种，其中最常用的是四层和七层负载均衡：

1）二层负载均衡

负载均衡服务器对外依然提供一个VIP（虚IP），集群中不同的机器采用相同IP地址，但机器的MAC地址不一样。当负载均衡服务器接收到请求之后，通过改写报文的目标MAC地址的方式将请求转发到目标机器实现负载均衡。

2）三层负载均衡

和二层负载均衡类似，负载均衡服务器对外依然提供一个VIP（虚IP），但集群中不同的机器采用不同的IP地址。当负载均衡服务器接收到请求之后，根据不同的负载均衡算法，通过IP将请求转发至不同的真实服务器。

3）四层负载均衡

四层负载均衡工作在OSI模型的传输层，由于在传输层，只有TCP/UDP协议，这两种协议中除了包含源IP、目标IP以外，还包含源端口号及目的端口号。四层负载均衡服务器在接收到客户端请求后，以后通过修改数据包的目标地址信息（IP+端口号）将流量转发到应用服务器。

4）七层负载均衡

七层负载均衡工作在OSI模型的应用层，应用层协议较多，常用http、radius、DNS等。七层负载就可以基于这些协议来负载。这些应用层协议中会包含很多有意义的内容。比如同一个Web服务器的负载均衡，除了根据IP加端口进行负载外，还可根据七层的URL、浏览器类别、语言来决定是否要进行负载均衡。

对于一般的应用来说，有了Nginx就够了。Nginx可以用于七层负载均衡。但是对于一些大的网站，一般会采用DNS+四层负载+七层负载的方式进行多层次负载均衡。

常用负载均衡工具

硬件负载均衡性能优越，功能全面，但价格昂贵，一般适合初期或者土豪级公司长期使用。因此软件负载均衡在互联网领域大量使用。常用的软件负载均衡软件有Nginx、LVS、HaProxy等。

4. (5分) 什么叫死锁? 常见的分布式系统死锁处理方法有哪些?

答: (来自百度)

死锁是指两个或两个以上的进程在执行过程中, 由于竞争资源或者由于彼此通信而造成的一种阻塞的现象, 若无外力作用, 它们都将无法推进下去。

有三种处理死锁的策略:

- 1) 预防死锁。限制请求, 保证前文提到的四个死锁条件中至少有一个不能发生, 从而预防死锁;
- 2) 避免死锁。如果结果状态是安全的, 就将资源动态地分给进程。如果至少有一个执行序列使所有的进程都能完成运行, 那么这个状态就是安全的;
- 3) 检测死锁和从死锁中恢复, 允许死锁发生, 然后发现并解除死锁。

死锁预防和避免采用一种悲观方法, 即认为死锁会经常发生并试图阻止或避免它。虽然死锁避免策略在集中式系统中广为应用, 并且有许多算法, 但是在分布式系统中很少使用。这是因为在分布式系统中没有全局时钟, 检查安全性状态会涉及到大量进程和资源的计算, 从而引起昂贵的开销。死锁检测和恢复使用乐观方法, 然而这种方法对于死锁发生频繁的应用程序可能并不有效。

四、程序流程设计题: (20分)

给出基于 Socket 的客户/服务器, 面向 TCP 网络通信编程的详细说明, 同时给出一个 UNIX/ Linux 的 Client/Sever 的程序设计算法流程图。

1. (10分) 如何进行服务器端编程设计?
2. (10分) 如何进行客户端编程设计? 如何设计服务器与客户的通信?

答: (同 2018.7)

客户服务器是一类按新的应用模式运行的分布式计算机系统。在这个应用模式中, 用户只关心完整地解决自己的应用问题, 而不关心这些应用问题由系统中哪台或哪几台计算机来完成。在 C/S 系统中, 能为应用提供服务 (如文件服务、打印服务、拷贝服务、图像服务、通信管理服务等)

的计算机或处理器, 当其被请求服务时就成为服务器。一台计算机可能提供多种服务, 一个服务

也可能要由多台计算机组合完成。与服务器相对, 提出服务请求的计算机或处理器在当时就是客

户机。从客户应用角度看, 这个应用的一部分工作在客户机上完成, 其他部分的工作则在 (一个

或多个) 服务器上完成。

在客户端, 1) 使用 socket 调用得到一个 socket 描述符 2) 使用 connect 向指定的服务器上的指定端口发起连接, 一旦连接成功返回, 就说明已经建立了与服务器的连接 3) 通过 socket 描述符进行读写操作。

在服务器端，1) 使用 socket 调用得到一个描述符 2) 使用 bind 调用将一个名字与 socket 描述符连接起来，对于 Internet 域就是将 Internet 地址联编到 socket 3) 服务端使用 Listen 调用指出等待服务请求队列的长度 4) 使用 accept 调用等待客户端发起连接，一旦有客户发出连接，accept 返回客户的地址信息，并返回一个新 socket 描述符，该描述符与原先的 socket 有相同的特性 5) 服务端使新的 socket 进行读写操作（一般的 socket 可能在 accept 返回后创建一个新的进程进行与客户的通信，父进程则再到 accept 调用处等待另一个连接。）

服务器的全部工作就是等候建立一个连接，然后用那个连接产生的 Socket 创建一个 InputStream 以及一个 OutputStream。开始连接之后，从 InputStream 读入的所有东西都会反馈给 OutputStream，直到接收到行 END 为止，最终关闭连接。

在客户机连接与服务器连接之后创建一个 OutputStream。在这时就可以发送一个文本文件。客户机在这时可创建一个 InputStream，从而开始对服务器的数据进行接收。

服务器与客户机可以使用同样的端口号，同时客户机利用本地主机地址连接位于同一台机器中的服务这个例子将以最简单的方式运用套接字对服务器和客户机进行操作。服务器的任务就是在建立连接，然后用那个连接产生的 Socket 创建一个 InputStream 以及一个 OutputStream。然后，它从 InputStream 读入的所有东西都会反馈给 OutputStream，直到接收到行中止 (END) 为止，最终将连接关闭。

UNIX/Linux 的 Client/Sever 的程序设计算法流程图（答案来自知识点）

TCP Connection Sequence

